

We've crossed a threshold where LLMs are no longer just faster tools but a fundamentally new computing paradigm—software 3.0—requiring us to rethink programming, infrastructure, hiring, and education around agent-native workflows.

The December Inflection Point

- December 2024 marked a stark transition: agentic tools stopped requiring corrections and began producing coherent, working code consistently
- This wasn't incremental improvement—it was a paradigm shift from 'helpful chunks' to 'vibe coding' where trust in the system became justified
- The shift demands re-evaluation: many dismissed AI in 2023 based on ChatGPT-era capabilities, but December models fundamentally changed what's tractable

I can't remember the last time I corrected it. I just trusted the system more and more, and then I was vibe coding.

Verifiability as the Unlock

- LLMs trained via reinforcement learning excel in verifiable domains (math, code) where outputs can be checked—this creates 'jagged' capability profiles
- Labs focus RL on economically valuable, verifiable tasks, so capability peaks where verification + lab interest align (e.g., chess improved dramatically when chess data entered GPT-4's pretraining)
- Paradox of jaggedness: Opus 4.7 refactors 100k-line codebases and finds zero-days, yet suggests walking to a car wash 50 meters away to wash your car
- Founders should target verifiable domains not yet prioritized by labs—if you can create RL environments with diverse datasets, fine-tuning becomes a tractable lever even without frontier lab focus

If you're in the circuits that were part of the RL, you fly. If you're out of the data distribution, you struggle.

What Humans Still Own

- Agents are interns: remarkable recall and execution, but humans remain in charge of aesthetics, judgment, taste, and oversight
- Example failure: agent tried matching Stripe and Google accounts by email address instead of persistent user IDs—agents miss obvious design flaws humans catch
- Humans design detailed specs and top-level architecture; agents fill in API details (keep_dims vs keep_dim, reshape vs permute) that humans no longer need to memorize
- Code quality issues persist: bloated, copy-pasted, brittle abstractions—models lack aesthetic cost functions in their RL, so taste remains human territory (for now)
- Simplification is hard for models: attempts to prompt for radical simplification (e.g., micro-GPT) feel like pulling teeth—outside RL circuits, models struggle

You can outsource your thinking, but you can't outsource your understanding.

Animals vs Ghosts

- LLMs are not animal intelligences—they're 'ghosts' shaped by data and reward functions, not intrinsic motivation, curiosity, or empowerment from evolution
- Yelling at them has no effect; they're statistical simulation circuits with RL bolted on top, creating jagged capability spikes
- This framing matters for competence: understanding what they are (and aren't) helps you deploy, evaluate, and trust them appropriately
- It's philosophical but practical—sets expectations for what's likely to work and how to modify systems when they don't

Software 3.0 Explained

- Software 1.0: explicit rules written in code. Software 2.0: learned weights via neural network training on datasets. Software 3.0: prompting LLMs that interpret context as a programmable computer
- Example: OpenClaw installation isn't a bash script—it's text you paste to an agent that intelligently adapts to your environment and debugs in the loop
- Menu generation evolution: built a full Vercel app with OCR and image generation, then realized the 3.0 version is just 'give Gemini the photo and ask it to overlay images'—the entire app becomes spurious
- This isn't about making programming faster; it's about automating general information processing that couldn't be coded before, like auto-generating knowledge bases from unstructured documents

Your neural network is doing more of the work. Your prompt or context is just the image, and the output is an image. There's no need for any app in between.

Vibe Coding vs Agentic Engineering

- Vibe coding raises the floor—everyone can build software. Agentic engineering preserves the quality bar of professional software while moving faster
- Agentic engineering is a discipline: coordinating powerful but stochastic, spiky agents without introducing vulnerabilities or sacrificing standards
- The ceiling is far higher than 10x—engineers skilled at agentic workflows achieve speed-ups well beyond that
- Hiring must evolve: stop giving puzzles; assign large projects like 'build a secure Twitter clone for agents, then use 10 Codex instances to try breaking it'—watch how candidates utilize tooling at scale

Agent-Native Infrastructure

- Current infrastructure is human-first: docs tell you what to do, not what to paste to your agent—this is friction
- Vision: describe a project to an LLM, and it deploys end-to-end without touching DNS, service configs, or deployment UIs
- Future workloads decompose into sensors (reading the world) and actuators (acting on it), with agent-legible data structures as the interface
- Agent-to-agent communication will handle logistics: your agent talks to my agent to schedule meetings, coordinate details, represent organizations

Why are people still telling me what to do? What is the thing I should copy-paste to my agent?

What Remains Worth Learning

- Understanding is the bottleneck: you can outsource thinking, but not understanding—someone must direct the agents and know why the work matters
- Tools that enhance understanding (LLM knowledge bases, synthetic data generation over fixed datasets) become critical for processing information and gaining insight
- Different projections onto the same information yield new understanding—reading articles while building a wiki from them creates prompts for deeper comprehension
- Humans remain uniquely in charge of understanding because LLMs don't excel at it—this makes you the director, and good directors need deep comprehension